

CS 431 Homework 03-04 SpMV on the CPU and GPU

David Haddad

November 23, 2025

1 Introduction

This report analyzes the performance of Sparse Matrix-Vector Multiplication (SpMV) implemented on both a multi-core CPU using OpenMP (Homework03) and a GPU using CUDA (Homework04). The primary purpose of these assignments was to explore different storage methods for the matrices – Coordinate (COO), Compressed Sparse Row (CSR), and ELLPACK (ELL) – and then evaluate how they perform under parallel execution of a specific matrix computation (multiplying a sparse matrix with a dense vector). These tests were conducted on a single matrix in `cant.mtx` using the Talapas cluster (which I now know has 28, not 32, CPU compute nodes and, of course, uses NVIDIA GPUs). These results are only accurate for this particular matrix under this specific Matrix-Vector computation. Other matrix manipulations and computations commonly found in linear algebra would exhibit differing performance across storage formats, since we are only really testing how the computer accesses memory during a single multiplication.

2 Implementation Strategy

2.1 CPU Parallelization (OpenMP)

For the CPU implementation, I focused on parallelizing the `convert_coo_to_csr` and `spmv_coo` functions.

- **Parallel Prefix Sum:** CSR conversion requires calculating row pointers, which in turn requires calculating a prefix sum. Homework02 gave the necessary insights to implement a prefix sum helper function to decrease this computational overhead.
- **Atomics vs. Locks:** After multiple tests with the COO SpMV kernel, the performance did not change. I experimented with using `omp_set_lock` and `#pragma omp atomic`. Neither the atomics nor the locks showed linear speedup. I decided to use the lock code for a variety of analyses. The overhead of the locks is clearly still visible as compared to CSR, which does not have such requirements.

2.2 GPU Parallelization (CUDA)

The GPU tested the CSR and ELL implementations using the CUDA platform. This portion of the assignment focuses more on memory coalescing (ELL explicitly and CSR through warps). Homework04 also involved the concepts of shared memory and how synchronization can be costly. In the naive implementation, shared memory is a huge bottleneck. Each thread must wait to access global memory before adding its value to the result vector. To minimize global memory writes, I used a parallel reduction to accumulate partial sums before writing the final dot product to the output vector. This tree-like reduction causes the threads to combine their results in a series of steps, only writing to global memory once per row.

3 Performance Analysis

3.1 CPU Scaling: COO vs. CSR

I tested the CPU implementation's scalability by varying the thread count from 1 to 28. I noted that when running on 32 threads, the expected speedup trend no longer held, as it did for 1-16 threads. This led me to discover that the compute nodes on Talapas have 28 cores per logical block/node. **Figure 1** shows how

the CSR format demonstrated near-perfect linear speedup. Unfortunately, the locks overhead in the COO format causes a plateau in speedup after we increase past four threads. The atomic version of the COO format was displaying confusing results and thus is not discussed in this report.

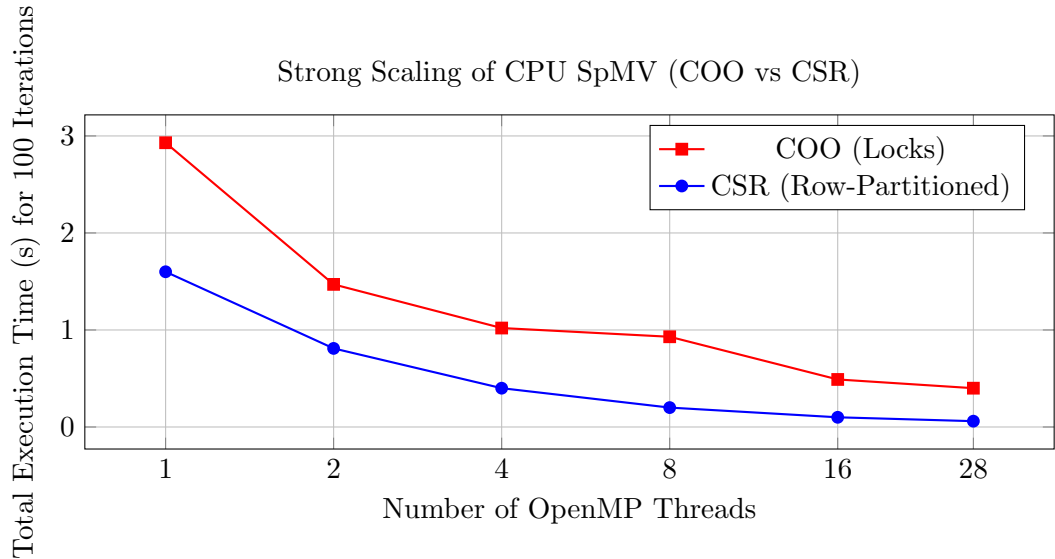


Figure 1: Execution time vs. thread count. CSR scales linearly, while COO struggles with lock overhead or atomic contention.

3.2 CPU vs. GPU Comparison

Comparing the best execution times for the CPU and the GPU highlights the latency differences between the two architectures. We expect to see a considerable speedup for the GPU, upwards of 10 times, but instead, the GPU is only roughly 1.5 times faster than the 28-core CPU CSR run. The most likely cause for this discrepancy is the matrix size. Modern GPUs are prepared to work with much larger datasets than our sample matrix `cant.mtx`. The time it takes to allocate and copy the input to the GPU is significantly longer than the time the GPU actually spends computing the result of our SpMV. This is why modern approaches consider more realistic situations in which the matrix is loaded into the GPU for hundreds or thousands of iterative computations (some SpMV, some not). This way, one can consider speedup and acceleration based on real-world computation scenarios rather than isolated test cases.

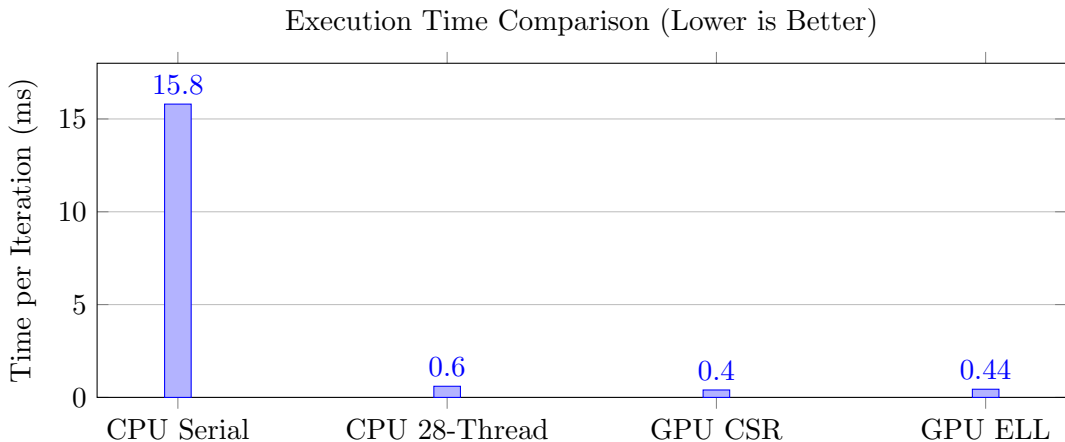


Figure 2: Per-iteration latency comparison. GPU CSR offers the lowest latency, marginally beating the highly parallel CPU CSR.

4 Conclusion

These two projects demonstrate the complex trade-offs programmers face when choosing storage format, algorithm structure, and computer architecture for large-scale code. Some formats, which may be easier to search, manipulate, and alter (COO), require atomic operations when computing Matrix-Vector products. CSR is superior for both the CPU and GPU because it uses memory more efficiently and allows rows to operate independently. This allows us as programmers to map the algorithm's structure neatly onto parallel constructs such as OpenMP pragmas and CUDA thread blocks/warps.